

CAHIER DES CHARGES DIRECTEUR

ELECTROSIM FLUTTER - NOUVEAU NOYAU

Architecture, solveur électrique, UX, scénarios de panne, migration, audits automatiques et protocole de développement autonome

Champ	Décision de référence
Objectif	Reconstruire ElectroSim sur Flutter/Dart avec un noyau électrique cohérent, testable et indépendant de l'interface.
Position de départ	L'ancienne application TypeScript est conservée comme référence fonctionnelle et de comparaison, mais aucun ancien schéma d'exemple ni ancien scénario de panne n'est migré automatiquement.
Bibliothèque d'exemples	Repart de zéro. Un exemple est obligatoirement un circuit sain, câblé et fonctionnel.
Bibliothèque de pannes	Repart de zéro. Une panne est un scénario autonome contenant directement un circuit défectueux.
Lien exemple/panne	Interdit. Aucune panne n'est injectée dans un schéma d'exemple.
Principe du moteur	Une seule source de vérité : CircuitState → TopologyEngine → SolverEngine → SimulationResult.
Cible prioritaire	Application installée hors ligne sur desktop et mobile. Web complémentaire.
Règle qualité	Aucun lot n'est validé sans tests automatiques + audit + preuve de résultat.

Version 1.0 - 27 septembre 2026

1. Objet du document et décision d'architecture

Ce cahier des charges constitue la référence de développement du nouvel ElectroSim. Il doit permettre de poursuivre le projet de manière autonome, séquentielle et contrôlée, en évitant la réintroduction des incohérences fonctionnelles accumulées dans l'ancienne architecture. Il ne promet pas l'absence absolue de défauts ; il impose en revanche un processus où les défauts doivent être détectés le plus tôt possible, reproduits automatiquement, corrigés puis couverts par un test de non-régression avant toute progression.

1.1 Décisions irréversibles de conception

- Flutter/Dart devient la plateforme principale de la nouvelle application.
- Le nouveau noyau électrique est indépendant de Flutter et ne dépend d'aucun widget, écran, service de navigation ou état visuel.
- Le Canvas n'effectue aucun calcul électrique : il ne fait qu'afficher SimulationResult et traduire les gestes de l'utilisateur en commandes métier.
- Les exemples et les scénarios de panne constituent deux bibliothèques indépendantes.
- Un scénario de panne contient son propre CircuitState défectueux et ses données pédagogiques privées ; il ne référence aucun exemple.
- Le moteur ne "sait" pas qu'un circuit est un exemple, un TP ou une panne. Il résout uniquement l'état électrique qui lui est fourni.
- Chaque résultat électrique affiché, chaque animation d'appareil, chaque mesure et chaque diagnostic doivent provenir d'un même SimulationResult versionné.
- Tout comportement corrigé doit recevoir un test automatique avant fermeture du défaut.
- Les anciennes données de bibliothèques ne sont pas réimportées. Les nouvelles bibliothèques sont reconstruites progressivement après validation du moteur.

1.2 Finalité recherchée

Le produit cible doit être à la fois un simulateur électrique robuste, un outil pédagogique, un environnement de câblage interactif et une plateforme de diagnostic. La qualité visuelle ne doit jamais masquer une incohérence physique : l'animation d'un composant n'est autorisée que si son état de fonctionnement est établi par le noyau électrique.

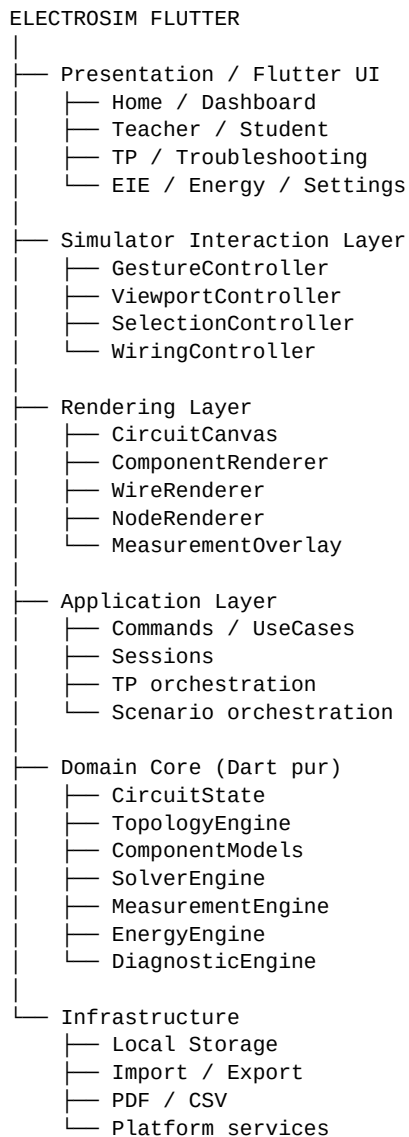
2. Périmètre fonctionnel cible

Domaine	Fonctions cibles	Principe de validation
Simulation	CC, AC monophasé, AC triphasé, PV, composants, conducteurs, nœuds, branches, mesures	Cas de référence numériques + invariants physiques + tests de topologie
Câblage	Ajout, déplacement, connexion, suppression, propriétés, routage de fils, zoom/pan	Tests de gestes, tests de modèle, scénarios E2E
Exemples	Bibliothèque de circuits sains et prêts à observer ou charger	Validation électrique automatique avant publication
Pannes	Bibliothèque autonome de circuits défectueux et vérités pédagogiques privées	Validation de cohérence du défaut + symptômes + réparabilité
TP câblage	Réalisation d'un circuit, mesures, mise en service, évaluation	Comparaison structurelle et/ou fonctionnelle selon activité
Recherche de dérangement	Diagnostic, mesures, hypothèses, réparation, remise en service	Le circuit réparé doit satisfaire les invariants et critères du scénario
Énergie	W, Wh, kWh, pertes, rendement, historique	Tests numériques tolérancés

Domaine	Fonctions cibles	Principe de validation
EIE	Explications contextuelles fondées sur topologie et résultats	Aucune affirmation sans preuve issue du moteur
Persistance	Sessions, sauvegardes multiples, reprise, export/import	Round-trip tests + migration de schéma de données
Distribution	Windows, macOS, Linux, Android, iOS ; Web complémentaire	Build automatisé + smoke tests par cible

3. Architecture cible

L'architecture est en couches strictes. Les dépendances ne doivent circuler que vers le bas. Le noyau électrique ne doit jamais importer de code UI.



3.1 Règles de dépendance

- Le package domain ne dépend que de Dart et de bibliothèques mathématiques explicitement approuvées.
- Le solveur ne dépend pas de Flutter, du stockage, du réseau, du Canvas ou de la navigation.
- Les écrans n'accèdent jamais directement à la base locale ; ils passent par des cas d'usage.
- Les renderers ne mutent jamais CircuitState.

- Une mutation du circuit passe par une commande validée, produit une nouvelle révision, puis déclenche un nouveau calcul.
- Un résultat de simulation porte obligatoirement l'identifiant et la révision du circuit qui l'a produit ; un résultat obsolète ne peut pas être affiché.

4. Modèle de domaine électrique

4.1 CircuitState - source unique de vérité

CircuitState est l'unique représentation autorisée de l'état électrique et structurel d'un circuit. Il est sérialisable, versionné, immuable ou traité comme tel, et indépendant du rendu.

```
CircuitState
- circuitId
- revision
- mode: dc | ac1 | ac3 | pv
- components[]
- connections[]
- sources[]
- settings
- metadata (non électrique)
```

```
ComponentInstance
- id
- modelType
- terminals[]
- parameters
- condition
- controlState
```

```
Connection
- id
- fromTerminalId
- toTerminalId
- conductorType
- phase / polarity metadata
- enabled
```

4.2 Séparation état électrique / état graphique

État électrique	État graphique
ID composant, modèle, bornes, paramètres, condition	Position x/y, rotation, taille visuelle
Connexion entre bornes	Points de routage du fil
État interrupteur/contact	Animation/position du levier
Résultat tension/courant	Couleur/effet visuel
Aucune notion de pixel	Aucune décision électrique

4.3 Contrats des composants

- Chaque type de composant déclare ses bornes et leurs rôles.
- Chaque modèle fournit sa contribution au système d'équations ou son comportement logique.
- Chaque composant expose ses limites nominales et ses états possibles.
- Les conditions défectueuses appartiennent au composant ou à la topologie réelle du scénario, pas à une couche d'injection externe.
- Un nouveau composant ne peut être ajouté au catalogue sans tests électriques, tests de sérialisation et test de rendu.

5. Nouveau moteur de résolution électrique

Le nouveau solveur doit être conçu comme un service mathématique déterministe. Pour une même entrée et une même version du moteur, il doit produire le même résultat à tolérance numérique définie.

5.1 Pipeline de calcul

```

CircuitState
  ↓ validation structurelle
TopologyEngine
  ↓ graphe électrique canonique
PreSolveDiagnostics
  ↓ erreurs bloquantes / avertissements
ComponentStamping
  ↓ matrice / équations
SolverEngine
  ↓ solution numérique
PostSolveValidation
  ↓ invariants physiques
MeasurementEngine
  ↓ valeurs instrumentables
DeviceStateEngine
  ↓ états de fonctionnement
EnergyEngine
  ↓ puissance / énergie
DiagnosticEngine
  ↓ explications traçables
SimulationResult

```

5.2 CC - analyse nodale modifiée

Le solveur CC doit utiliser une formulation nodale rigoureuse, idéalement MNA (Modified Nodal Analysis), afin de traiter de manière cohérente les résistances, sources de tension/courant, interrupteurs, circuits ouverts, courts-circuits contrôlés, branches parallèles et réseaux comportant plusieurs nœuds.

- Tolérance numérique globale configurable et centralisée.
- Détection de matrice singulière avec diagnostic explicite plutôt qu'un résultat arbitraire.
- Détection des nœuds flottants.
- Traitement contrôlé des interrupteurs ouverts/fermés et résistances extrêmes sans "valeurs magiques" dispersées.
- Tests de Kirchhoff KCL/KVL sur chaque résultat résolu.

5.3 AC monophasé

- Représentation complexe des tensions, courants et impédances.
- Calcul de P, Q, S, facteur de puissance et angle de phase.
- Modèles R, L, C et charges équivalentes explicitement documentés.
- Fréquence portée par le contexte de simulation.
- Les mesures RMS et phases doivent être distinguées des valeurs instantanées si celles-ci sont ajoutées plus tard.

5.4 AC triphasé

Le triphasé ne doit pas être simulé comme « une phase multipliée par trois ». L1, L2, L3 et N doivent être représentés explicitement pour permettre les déséquilibres, pertes de phase, inversion de phases, erreurs de câblage et neutres manquants.

- Topologies étoile et triangle.
- Ordre des phases explicite.
- Charges équilibrées et déséquilibrées.
- Courants de ligne et de phase.
- Présence ou absence du neutre.
- Inversion de deux phases représentée par le câblage réel.

5.5 PV

- Modèle source PV clairement séparé du modèle de réseau.
- Paramètres d'irradiance/température uniquement si physiquement pris en charge ; sinon ne pas afficher des grandeurs fictives.
- Onduleur modélisé avec conditions d'entrée/sortie, rendement, limites et état de fonctionnement.
- Une panne d'onduleur doit être un état de composant ou un scénario topologique réel produisant des sorties conformes au modèle, jamais un simple indicateur UI.

6. SimulationResult et cohérence globale

```
SimulationResult
- circuitId
- circuitRevision
- engineVersion
- status: solved | unsolved | invalid
- nodeVoltages
- branchCurrents
- componentResults
- sourceResults
- measurements
- energySnapshot
- topologyFindings
- diagnostics
- warnings
- numericalResiduals
- solveDuration
```

- Un résultat n'est utilisable que si circuitRevision correspond à la révision actuellement affichée.
- Les animations lisent componentResults ; elles n'infèrent pas l'état depuis un bouton ou un écran.
- Le multimètre et les instruments lisent MeasurementEngine, jamais directement l'état visuel.
- L'EIE cite les diagnostics structurés et les preuves disponibles dans SimulationResult.
- Les exports et historiques énergétiques enregistrent la version du moteur pour assurer la traçabilité.

7. Nouvelle bibliothèque d'exemples

La bibliothèque d'exemples repart de zéro. Un exemple est un circuit fonctionnel et validé. Il peut servir d'observation, de modèle, de point de départ ou de support de cours, mais jamais de support interne à une injection de panne.

```
ExampleCircuit
- id
- title
- description
- electricalMode
- circuitState
- expectedOperatingStates
- expectedMeasurements
- tags
```

- version
- validationSignature

PORTE DE QUALITÉ - Publication d'un exemple

- Validation du schéma de données.
- Résolution sans erreur bloquante.
- Résidus numériques sous seuil.
- KCL/KVL respectées dans les tolérances définies.
- Aucun composant critique hors plage si l'exemple est déclaré « fonctionnement normal ».
- Valeurs attendues de référence comparées automatiquement.
- Smoke test de chargement dans le Canvas.

Critère de passage : Tous les contrôles doivent être verts. Un exemple non résolu ou ambigu ne peut pas être publié.

Blocage automatique : Publication interdite si un test échoue.

8. Nouvelle bibliothèque de scénarios de panne

Une panne est désormais un scénario complet et autonome. Le circuit fourni à l'élève est déjà défectueux. Les défauts peuvent être topologiques, liés aux composants, aux réglages ou à une combinaison contrôlée de ces catégories.

```
FaultScenario
- id
- title
- electricalMode
- faultyCircuitState
- studentBrief
- teacherTruth (privé)
  - rootCauses[]
  - expectedSymptoms[]
  - expectedMeasurements[]
  - acceptableRepairs[]
  - evaluationCriteria
- difficulty
- estimatedDuration
- version
- validationSignature
```

8.1 Typologie des défauts autorisés

Catégorie	Exemples	Représentation
Câblage	conducteur absent, mauvaise borne, inversion, court-circuit	connections[] réelles
Composant	filament ouvert, bobine coupée, moteur bloqué, contact collé	condition du composant
Réglage	protection mal réglée, paramètre incorrect	paramètre réel du composant
Source	sous-tension, phase absente, polarité inversée	état/paramètres de la source
Multiple	deux défauts compatibles pour niveau avancé	combinaison explicite, jamais injection cachée

PORTE DE QUALITÉ - Publication d'un scénario de panne

- Le scénario doit être résolu ou diagnostiqué comme volontairement non résoluble selon sa nature.
- La panne annoncée doit provoquer les symptômes attendus.
- Les mesures pédagogiques attendues doivent être reproductibles dans la tolérance.
- Le scénario doit disposer d'au moins une réparation valide.
- La réparation de référence doit remettre le circuit en état fonctionnel.
- Les données teacherTruth doivent être absentes de la charge élève.
- Le scénario ne doit contenir aucune référence à un ExampleCircuit.

Critère de passage : Tous les tests passent et le scénario est réparé automatiquement par au moins un script de référence.

Blocage automatique : Toute fuite teacherTruth ou dépendance à un exemple bloque la publication.

9. Vérification des réparations

La validation d'une réparation ne doit pas dépendre de la suppression d'un identifiant de panne. Le circuit réparé est recalculé comme n'importe quel circuit.

```

StudentCircuit after repair
    ↓
TopologyEngine
    ↓
SolverEngine
    ↓
VerificationEngine
    ↓
- fonctionnement attendu ?
- sécurité respectée ?
- mesures dans tolérance ?
- causes racines supprimées ?
- aucune nouvelle anomalie critique ?

```

- Une solution alternative techniquement correcte peut être acceptée si les critères du scénario l'autorisent.
- Les critères d'évaluation distinguent résultat final, démarche, mesures, hypothèses et remise en service.
- Le système ne révèle jamais automatiquement la cause racine avant la fin si le mode pédagogique ne l'autorise pas.

10. UX et moteur graphique Flutter

10.1 Gestes standards

Action utilisateur	Comportement requis
Clic sur composant	Sélection sans déplacement.
Appui long + glisser	Déplacement du composant avec prévisualisation.
Clic sur borne	Démarre une connexion si le contexte l'autorise.
Déplacement vers borne	Surligne les cibles compatibles.
Clic borne cible	Valide la connexion.
Double-clic fil	Action contextuelle nœud/édition si autorisée.
Double-clic composant	Propriétés / remplacer / supprimer selon contexte.
Pinch / molette	Zoom centré sur le point d'interaction.
Pan	Déplacement de la vue sans altérer le circuit.

10.2 Architecture du Canvas

```

CircuitCanvas
├── ViewportTransform
├── GridLayer
├── WireLayer
├── ComponentLayer
├── TerminalLayer
├── SelectionLayer
├── MeasurementLayer
├── DiagnosticHighlightLayer
└── InteractionOverlay

```

- Le hit-testing doit être centralisé pour éviter des gestionnaires concurrents.
- Les coordonnées monde et écran doivent être séparées.
- Le zoom ne change jamais les coordonnées électriques/logiques.
- Les visuels de palette et les visuels déposés doivent utiliser la même identité graphique ou une variante explicitement définie, afin d'éviter les incohérences de représentation.
- Le bouton « voir plus de composants » et tout contrôle similaire doivent avoir des tests de tap répétés et de latence.

11. Responsive et profils d'usage

Profil	Largeur indicative	Disposition
Compact	< 600 px	Simulateur prioritaire, panneaux en tiroirs, barre d'actions basse.
Medium	600–1000 px	Simulateur + panneau secondaire escamotable.
Expanded	> 1000 px	Palette + simulateur + panneau de propriétés/EIE selon contexte.

- Les écrans élève et professeur doivent être testés dans les trois profils.
- Aucun élément essentiel ne doit être hors écran sans mécanisme de défilement ou tiroir.
- La fiche de diagnostic n'existe dans l'interface élève que pendant une activité de recherche de dérangement.
- Les actions non pertinentes pour le rôle ou l'activité doivent être absentes, pas simplement désactivées sans explication.

12. Persistance, versions et migrations de données

- Tous les formats persistés portent schemaVersion.
- Toute évolution de format dispose d'une migration automatisée ou d'une décision explicite d'incompatibilité.
- Les sauvegardes utilisateur sont distinctes des bibliothèques système.
- « Enregistrer » crée/met à jour une sauvegarde locale ; PDF/CSV sont des exports séparés.
- « Ouvrir » affiche une liste de sauvegardes et ne charge jamais arbitrairement la dernière sauvegarde.
- Les écritures utilisent un mécanisme atomique ou transactionnel pour éviter un fichier partiellement écrit.
- Un test round-trip serialize → deserialize → compare est obligatoire pour chaque type persistant.

13. Confidentialité pédagogique et séparation professeur/élève

- Les vérités de panne, corrections et résultats de référence ne doivent jamais être inclus dans le payload élève lorsqu'ils ne sont pas nécessaires.
- Les vues professeur et élève utilisent le même noyau mais des politiques d'accès différentes.

- Une activité terminée côté élève devient en lecture seule ; seules les données de résultat restent consultables.
- La supervision professeur doit être fondée sur un état de session explicite et transactionnel, pas sur l'état visuel de l'écran élève.
- Les transitions started → submitted → evaluated → closed doivent être testées comme machine à états.

14. Stratégie générale de tests automatiques

Flutter recommande de combiner tests unitaires, tests de widgets et tests d'intégration. Le projet appliquera cette pyramide, complétée par des tests mathématiques du solveur, des tests de propriétés et des golden tests pour les rendus critiques.

Niveau	But	Exécution
Analyse statique	Erreurs de type, règles de lint, dépendances interdites	Chaque commit
Unitaires domaine	Mathématiques, modèles, topologie, commandes	Chaque commit
Tests de propriétés	Invariants physiques sur familles de circuits générés	Chaque PR + nuit
Widget tests	Comportement UI isolé, boutons, formulaires, responsive	Chaque PR
Golden tests	Régressions visuelles contrôlées	PR touchant UI
Intégration	Parcours complets de l'application	Chaque PR ciblée + release
E2E plateforme	Build réel et gestes principaux	Release candidate
Performance	Temps solveur, FPS, mémoire, taille des circuits	Nuit + release
Persistance	Round-trip, corruption, reprise	Chaque PR stockage
Sécurité pédagogique	Absence de teacherTruth côté élève	Chaque PR scénarios/TP

14.1 Invariants électriques obligatoires

- KCL : somme algébrique des courants d'un nœud ≈ 0 dans la tolérance.
- KVL : somme des tensions sur une maille de référence ≈ 0 quand applicable.
- Conservation de puissance dans les modèles où elle s'applique, avec pertes explicitement modélisées.
- Un interrupteur ouvert idéal ne transporte pas de courant au-delà de la tolérance définie.
- Un conducteur idéal relie des potentiels compatibles selon le modèle utilisé.
- Une source ne peut pas produire silencieusement un état impossible : le solveur doit signaler les contradictions.
- Les unités de toutes les valeurs numériques doivent être explicites dans les API.

14.2 Cas canoniques initiaux du solveur

Cas	Attendu principal	Type
DC-001 : source 24 V + R=12 Ω	I=2 A, UR=24 V	Automatique
DC-002 : R1=10 Ω + R2=20 Ω en série sous 30 V	I=1 A, U1=10 V, U2=20 V	Automatique
DC-003 : 10 Ω // 10 Ω sous 10 V	Req=5 Ω , Itotal=2 A, 1 A par branche	Automatique
DC-004 : interrupteur ouvert	courant nul dans la branche	Automatique
DC-005 : nœud flottant	diagnostic explicite	Automatique
DC-006 : court-circuit source idéale	état invalide/limité selon modèle, jamais valeur arbitraire	Automatique
AC1-001 : charge résistive	$Q \approx 0$, $\cos \varphi \approx 1$	Automatique
AC1-002 : charge inductive	$Q > 0$ et déphasage cohérent	Automatique
AC3-001 : charge étoile équilibrée	courants équilibrés	Automatique
AC3-002 : perte L2	état triphasé dégradé détecté	Automatique
AC3-003 : inversion de deux phases	ordre de phases modifié	Automatique

Cas	Attendu principal	Type
FAULT-001 : fil réellement absent	symptômes issus de la topologie	Automatique
FAULT-002 : composant open condition	courant et tensions cohérents avec circuit ouvert	Automatique
REPAIR-001 : réparation correcte	retour aux critères de fonctionnement	Automatique

15. Pipeline CI/CD et portes de qualité

```

COMMIT
↓
format + analyze
↓
unit tests
↓
domain invariants
↓
widget tests
↓
serialization tests
↓
architecture checks
↓
PR BUILD
↓
integration tests
↓
golden tests if UI changed
↓
solver benchmark delta
↓
security/pedagogy leakage checks
↓
MERGE
↓
nightly property tests + large-circuit tests
↓
RELEASE CANDIDATE
↓
desktop/mobile builds + smoke E2E + audit report

```

- Aucune fusion avec analyse statique rouge.
- Aucune fusion avec test de non-régression rouge.
- Aucune baisse de couverture du noyau sans justification documentée.
- Aucun benchmark critique ne peut régresser au-delà du seuil accepté sans revue explicite.
- Toute modification du format de données doit fournir tests de migration/compatibilité.
- Toute modification du solveur doit produire un rapport de comparaison sur le corpus canonique.

16. Structure recommandée du dépôt

```

electrosim_flutter/
├── apps/
│   └── electrosim/
├── packages/
│   ├── electrosim_domain/
│   ├── electrosim_solver_dc/
│   ├── electrosim_solver_ac/
│   ├── electrosim_energy/
│   ├── electrosim_diagnostics/
│   ├── electrosim_scenarios/
│   ├── electrosim_storage/
│   └── electrosim_ui_kit/
└── test_vectors/

```

```

|   |   | dc/
|   |   | ac1/
|   |   | ac3/
|   |   | pv/
|   |   | faults/
|   |   |
|   |   | tools/
|   |   | validate_catalog.dart
|   |   | compare_solver_results.dart
|   |   | generate_audit_report.dart
|   |   | architecture_guard.dart
|   |   |
|   |   | integration_test/
|   |   |
|   |   | docs/
|   |   |
|   |   | ci/

```

17. Protocole obligatoire de travail autonome

Chaque intervention autonome doit suivre exactement la séquence ci-dessous. Le principe est : une modification minimale, une preuve automatique, un artefact testable, puis seulement l'étape suivante.

1. Lire le dernier état du dépôt et le dernier rapport d'audit ; identifier le dernier jalon validé.
2. Exécuter la baseline de tests avant toute modification. Si elle est rouge, ne pas empiler une nouvelle fonctionnalité : isoler la régression existante.
3. Définir une seule unité de travail ayant un résultat observable et un critère d'acceptation automatique.
4. Ajouter ou mettre à jour le test qui prouve le comportement cible avant ou en même temps que le code.
5. Implémenter la plus petite modification satisfaisant ce test.
6. Exécuter tests unitaires ciblés, puis suite du package, puis suite globale selon la criticité.
7. Exécuter l'audit architecture/données/performance correspondant au domaine modifié.
8. Générer un rapport de changement : fichiers, comportement, tests exécutés, résultats, risques résiduels.
9. Produire un artefact exécutable/testable seulement si toutes les portes obligatoires sont vertes.
10. Marquer le jalon comme candidat ; ne le déclarer validé physiquement qu'après contrôle utilisateur si un contrôle humain est requis.

17.1 Règles d'arrêt immédiat

- Régression d'un test électrique canonique.
- Résultat numérique non déterministe non expliqué.
- Divergence entre circuitRevision et SimulationResult affiché.
- Fuite d'une donnée teacherTruth vers un contexte élève.
- Perte ou corruption de sauvegarde pendant un test round-trip.
- Crash reproductible sur un parcours principal.
- Introduction d'une dépendance UI dans le noyau métier.
- Une correction nécessite de masquer le problème par une condition spéciale liée à un identifiant de scénario ou d'exemple.

18. Plan de réalisation étape par étape

F0 - Sanctuarisation et laboratoire de référence

Travaux :

- Geler une copie de référence de l'application actuelle.
- Exporter uniquement les connaissances utiles : types de composants, règles métiers documentées, cas de test connus.

- Créer le dépôt Flutter neuf et la convention de version.
- Créer le corpus initial de circuits de référence indépendant des anciennes bibliothèques.

Livrables :

- Dépôt Flutter vide mais compilable.
- Rapport de baseline.
- Catalogue de cas de test électriques.

PORTE DE QUALITÉ - F0 - Sanctuarisation et laboratoire de référence

- flutter analyse sans erreur
- tests squelette verts
- hash/version de la référence enregistrés

Critère de passage : Tous les contrôles listés passent et le rapport d'audit est archivé.

Blocage automatique : Aucune migration de schéma/panne historique.

F1 - Socle Dart pur et contrats de domaine

Travaux :

- Créer IDs, unités, types, modes électriques, erreurs structurées.
- Créer CircuitState, ComponentInstance, Terminal, Connection.
- Créer sérialisation versionnée.
- Interdire toute dépendance Flutter dans domain.

Livrables :

- Package electrosim_domain.
- Tests de sérialisation et égalité.

PORTE DE QUALITÉ - F1 - Socle Dart pur et contrats de domaine

- round-trip 100 %
- architecture_guard vert
- couverture domaine ≥ 90 % sur primitives critiques

Critère de passage : Tous les contrôles listés passent et le rapport d'audit est archivé.

Blocage automatique : Pas de Canvas ni d'écran métier avant stabilisation des contrats.

F2 - TopologyEngine

Travaux :

- Compiler bornes/connexions en graphe canonique.
- Détecter nœuds, composantes connexes, conducteurs ouverts, nœuds flottants.
- Créer diagnostics pré-solve.

Livrables :

- TopologyGraph.
- TopologyFinding structurés.

PORTE DE QUALITÉ - F2 - TopologyEngine

- corpus topologique 100 % vert
- tests propriété connectivité
- aucune mutation de CircuitState

Critère de passage : Tous les contrôles listés passent et le rapport d'audit est archivé.

Blocage automatique : Aucun calcul de tension/courant dans cette phase.

F3 - Solveur CC

Travaux :

- Implémenter MNA.
- Résistances, sources, interrupteurs, connexions.
- Résidus numériques et diagnostics de singularité.
- KCL/KVL automatiques.

Livrables :

- SolverDC.
- Corpus DC-001.. étendu.

PORTE DE QUALITÉ - F3 - Solveur CC

- tous cas canoniques verts
- résidu < seuil
- benchmark baseline enregistré

Critère de passage : Tous les contrôles listés passent et le rapport d'audit est archivé.

Blocage automatique : Ne pas poursuivre vers AC si le CC n'est pas stable.

F4 - MeasurementEngine et états appareils CC

Travaux :

- Créer sondes virtuelles et mesures.
- Calculer état de fonctionnement depuis résultats.
- Créer limites nominales et warnings.

Livrables :

- MeasurementResult.
- ComponentOperatingState.

PORTE DE QUALITÉ - F4 - MeasurementEngine et états appareils CC

- tests mesure V/A/ Ω selon modes permis

- aucun composant visuel ne décide son état

Critère de passage : Tous les contrôles listés passent et le rapport d'audit est archivé.

Blocage automatique : Les mesures invalides doivent renvoyer une erreur explicite.

F5 - Solveur AC1

Travaux :

- Complexes/phasors.
- R/L/C et charges usuelles.
- P/Q/S/cos ϕ .

Livrables :

- SolverAC1.
- Corpus AC1.

PORTE DE QUALITÉ - F5 - Solveur AC1

- cas résistif/inductif/capacitif verts
- unités et angles testés

Critère de passage : Tous les contrôles listés passent et le rapport d'audit est archivé.

Blocage automatique : Aucun affichage de grandeur non réellement calculée.

F6 - Solveur AC3

Travaux :

- L1/L2/L3/N explicites.
- Étoile/triangle.
- Déséquilibre, perte de phase, ordre de phase.

Livrables :

- SolverAC3.
- Corpus triphasé.

PORTE DE QUALITÉ - F6 - Solveur AC3

- tests perte de phase/inversion verts
- courants équilibrés cas nominal

Critère de passage : Tous les contrôles listés passent et le rapport d'audit est archivé.

Blocage automatique : Interdit de remplacer le calcul par une multiplication x3.

F7 - PV et énergie

Travaux :

- Modèles PV définis.
- Onduleur avec limites/états.
- EnergyEngine W/Wh/kWh/pertes/rendement.

Livrables :

- SolverPV ou module PV.
- EnergyEngine.

PORTE DE QUALITÉ - F7 - PV et énergie

- tests puissance/énergie temporelle
- panne onduleur arrête réellement la sortie selon modèle

Critère de passage : Tous les contrôles listés passent et le rapport d'audit est archivé.

Blocage automatique : Pas de valeurs décoratives ou arbitraires.

F8 - Canvas Flutter et interaction de base

Travaux :

- Créer viewport, rendu composants/fils/bornes.
- Hit-testing centralisé.
- Sélection, drag, zoom, pan, wiring.

Livrables :

- SimulatorCanvas fonctionnel sans logique électrique cachée.

PORTE DE QUALITÉ - F8 - Canvas Flutter et interaction de base

- widget tests gestes
- golden tests 3 tailles
- 60 fps cible sur scène nominale

Critère de passage : Tous les contrôles listés passent et le rapport d'audit est archivé.

Blocage automatique : Le Canvas consomme CircuitState/SimulationResult uniquement.

F9 - Design system et responsive

Travaux :

- Créer tokens, composants UI, navigation.
- Profils compact/medium/expanded.
- Palette cohérente et appareils réalistes.

Livrables :

- UI kit.
- Écrans structurels.

PORTE DE QUALITÉ - F9 - Design system et responsive

- goldens desktop/tablette/mobile
- audit accessibilité
- aucun overflow

Critère de passage : Tous les contrôles listés passent et le rapport d'audit est archivé.

Blocage automatique : La beauté ne justifie jamais une divergence fonctionnelle.

F10 - Bibliothèque d'exemples neuve

Travaux :

- Créer outil de création/validation.
- Ajouter quelques exemples simples seulement.
- Signer la validation.

Livrables :

- ExampleRepository neuf.

PORTE DE QUALITÉ - F10 - Bibliothèque d'exemples neuve

- validator 100 % vert
- aucune dépendance FaultScenario

Critère de passage : Tous les contrôles listés passent et le rapport d'audit est archivé.

Blocage automatique : Commencer petit : qualité avant quantité.

F11 - Bibliothèque de pannes neuve

Travaux :

- Créer FaultScenario autonome.
- Créer teacherTruth privé.
- Créer scénarios de câblage incorrect et composants défectueux.

Livrables :

- FaultScenarioRepository neuf.

PORTE DE QUALITÉ - F11 - Bibliothèque de pannes neuve

- réparabilité testée
- teacherTruth leak test vert
- aucun exampleId

Critère de passage : Tous les contrôles listés passent et le rapport d'audit est archivé.

Blocage automatique : Une panne est un circuit défectueux autonome.

F12 - TP câblage et recherche de dérangement

Travaux :

- Machine à états TP.
- Publication, démarrage, validation, clôture.
- Fiche diagnostic uniquement élève/RD.
- Réparation recalculée par le solveur.

Livrables :

- TPEngine.
- TroubleshootingWork.

PORTE DE QUALITÉ - F12 - TP câblage et recherche de dérangement

- E2E prof→élève→validation→résultat
- TP terminé lecture seule
- score généré

Critère de passage : Tous les contrôles listés passent et le rapport d'audit est archivé.

Blocage automatique : Un seul TP actif si cette règle est conservée.

F13 - EIE fondé sur preuves

Travaux :

- Construire explications à partir de Findings/SimulationResult.
- Ajouter localiser/surligner.
- Aucun diagnostic inventé.

Livrables :

- DiagnosticEngine/EIE.

PORTE DE QUALITÉ - F13 - EIE fondé sur preuves

- chaque conseil possède evidenceIds
- tests de non-contradiction

Critère de passage : Tous les contrôles listés passent et le rapport d'audit est archivé.

Blocage automatique : Si la preuve est insuffisante, l'EIE doit le dire.

F14 - Sauvegardes, import/export et reprise

Travaux :

- Sauvegardes multiples.
- Liste Ouvrir.
- Autosave contrôlé.
- CSV/PDF.

Livrables :

- StorageRepository.

- Exports.

PORTE DE QUALITÉ - F14 - Sauvegardes, import/export et reprise

- round-trip
- recovery after interrupted write
- compat schema tests

Critère de passage : Tous les contrôles listés passent et le rapport d'audit est archivé.

Blocage automatique : Enregistrer ≠ exporter PDF.

F15 - Distribution et qualification multi-plateforme

Travaux :

- Build desktop/mobile.
- Permissions minimales.
- Tests hors-ligne.
- Packaging release.

Livrables :

- Installateurs / builds.
- Matrice de qualification.

PORTE DE QUALITÉ - F15 - Distribution et qualification multi-plateforme

- smoke E2E par OS cible
- offline test
- crash-free scenario pack

Critère de passage : Tous les contrôles listés passent et le rapport d'audit est archivé.

Blocage automatique : Web reste complémentaire tant que la parité n'est pas démontrée.

F16 - Reconstruction progressive du catalogue

Travaux :

- Ajouter exemples et pannes uniquement par lots validés.
- Chaque ajout passe le validator.
- Versionner le contenu.

Livrables :

- Catalogue pédagogique qualifié.

PORTE DE QUALITÉ - F16 - Reconstruction progressive du catalogue

- 100 % items signés/validés
- aucun contenu legacy caché

Critère de passage : Tous les contrôles listés passent et le rapport d'audit est archivé.

Blocage automatique : Aucune importation en masse non auditée.

19. Rapports d'audit automatiques à produire

Rapport	Contenu minimal	Fréquence
audit_static.json	analyse, lint, dépendances interdites, TODO critiques	Chaque PR
audit_solver.json	cas passés/échoués, résidus, temps, version moteur	Chaque changement solveur
audit_catalog.json	exemples/pannes valides, erreurs, références interdites	Chaque changement contenu
audit_ui.json	widget tests, goldens, overflow, tailles	Chaque changement UI
audit_storage.json	round-trip, migrations, récupération	Chaque changement données
audit_security.json	teacherTruth leakage, fichiers sensibles	Chaque changement TP/panne
audit_performance.json	temps solveur p50/p95, FPS, mémoire, gros circuits	Nightly + release
audit_release.md	synthèse complète, versions, exceptions, risques	Chaque release candidate

20. Definition of Done (DoD)

- Le comportement demandé est couvert par au moins un test automatique pertinent.
- Les tests existants restent verts.
- Le code respecte les frontières d'architecture.
- Le changement ne crée aucune dépendance entre exemples et pannes.
- Les données persistent et se relisent correctement si elles sont concernées.
- Le rendu responsive est vérifié si l'UI est concernée.
- La performance est comparée à la baseline si le solveur/Canvas est concerné.
- Le rapport d'audit associé est généré et archivé.
- Le défaut corrigé reçoit un test de non-régression portant son identifiant.
- La documentation du contrat ou du format est mise à jour si nécessaire.

21. Indicateurs de qualité

Indicateur	Cible initiale	Blocage release
Tests canoniques solveur	100 %	Oui
Tests catalogue	100 %	Oui
TeacherTruth leakage	0 occurrence	Oui
Erreurs analyse statique	0	Oui
Crashes E2E critiques	0	Oui
Round-trip sauvegardes	100 %	Oui
Couverture noyau critique	>= 90 % lignes + tests de propriétés	Oui si baisse injustifiée
Résidus numériques	Sous seuil par mode	Oui
Overflow UI profils supportés	0 sur écrans critiques	Oui
Solve duration	Baseline puis seuils par taille de circuit	Oui si forte régression

22. Exigences non fonctionnelles

- Hors ligne : les fonctions principales de simulation, TP local, exemples, pannes, mesures et sauvegardes ne dépendent d'aucun service Internet.
- Déterminisme : mêmes entrées + même version moteur → mêmes sorties dans la tolérance.
- Traçabilité : toute simulation exportable inclut engineVersion et circuitRevision.

- Réactivité : les calculs lourds ne bloquent pas durablement le thread UI ; utiliser isolate lorsque le profilage le justifie sur plateformes compatibles.
- Accessibilité : tailles de texte, contrastes, navigation clavier desktop et sémantique des contrôles principaux.
- Internationalisation : les chaînes UI ne doivent pas être codées en dur dans le noyau.
- Robustesse : erreurs explicites et récupérables ; pas de “catch silencieux” qui fait croire à un succès.
- Observabilité locale : journaux de diagnostic exportables sans données pédagogiques privées non nécessaires.

23. Anti-patterns interdits

- Condition spéciale du type « si scenarioId == X alors forcer telle valeur ».
- Un widget modifie directement une tension, un courant ou un état électrique.
- Un exemple contient un faultId ou un scénario de panne contient un exampleId.
- Le moteur retourne 0 par défaut lorsque le calcul a échoué sans signaler l'erreur.
- Copie divergente du circuit conservée par plusieurs modules.
- Utilisation d'un booléen UI comme preuve qu'un composant fonctionne.
- Données professeur envoyées à l'élève puis simplement masquées visuellement.
- Correction d'un bug sans test de reproduction.
- Ajout d'un composant sans modèle électrique documenté.
- Optimisation prématurée native/Rust avant mesure de performance.

24. Scénarios d'acceptation système prioritaires

ID	Scénario
SA-01 Câblage simple	Créer source + interrupteur + lampe, câbler, fermer interrupteur, observer lampe et mesures cohérentes.
SA-02 Fil absent	Retirer le conducteur retour : lampe éteinte, courant nul, diagnostic circuit ouvert.
SA-03 Mauvaise borne	Brancher volontairement une borne incorrecte : résultat calculé depuis la topologie, pas depuis une règle de scénario.
SA-04 Panne onduleur	Charger un scénario autonome avec onduleur défectueux : sortie conforme à condition de panne ; appareil aval ne peut pas rester artificiellement fonctionnel.
SA-05 Réparation alternative	Réparer un scénario par une solution valide différente de la correction de référence et obtenir une validation si les critères fonctionnels sont satisfaits.
SA-06 TP terminé	Soumettre/terminer : côté élève lecture seule ; côté professeur progression terminée et score disponible.
SA-07 Mobile	Effectuer ajout composant, câblage, mesure et retour accueil sans zone inaccessible.
SA-08 Sauvegarde multiple	Créer deux sauvegardes, fermer, rouvrir, choisir explicitement l'une des deux et retrouver exactement son état.
SA-09 Exemple sain	Charger un exemple : aucune panne cachée, fonctionnement nominal vérifié par validator.
SA-10 Isolement contenu	Inspection du payload élève : aucune teacherTruth, aucune correction privée.

25. Commandes de contrôle recommandées

Les commandes exactes pourront évoluer avec le dépôt, mais la CI doit fournir des scripts stables équivalents aux intentions suivantes :

```
# Qualité de base
dart format --output=none --set-exit-if-changed .
flutter analyze
flutter test

# Tests ciblés
flutter test packages/electrosim_domain/test
flutter test packages/electrosim_solver_dc/test
flutter test packages/electrosim_solver_ac/test

# Intégration
flutter test integration_test

# Audits projet
dart run tools/architecture_guard.dart
dart run tools/validate_catalog.dart
dart run tools/compare_solver_results.dart test_vectors/
dart run tools/generate_audit_report.dart

# Couverture
flutter test --coverage
```

26. Références techniques Flutter utilisées pour ce cahier

Référence	URL	Utilisation
Supported deployment platforms	https://docs.flutter.dev/reference/supported-platforms	Flutter 3.47 : Android, iOS, Windows, macOS, Linux et Web pris en charge selon matrice officielle.
Testing Flutter apps	https://docs.flutter.dev/testing/overview	Pyramide tests unitaires, widgets et intégration.
Integration testing	https://docs.flutter.dev/testing/integration-tests	Package integration_test et exécution sur cible.
Concurrency and isolates	https://docs.flutter.dev/perf/isolates	Déport des calculs lourds hors isolate UI lorsque nécessaire.
Web support	https://docs.flutter.dev/platform-integration/web	Flutter Web pour applications riches et interactives ; cible complémentaire.

27. Contrat directeur pour la suite du projet

À partir de ce cahier des charges, la règle de conduite du projet est la suivante : aucune fonctionnalité historique n'est reprise par simple copie. Chaque capacité est réintroduite dans le nouveau noyau uniquement après définition de son contrat, création de tests, implémentation minimale, audit automatique et validation du jalon. Les exemples sont reconstruits comme circuits sains. Les pannes sont reconstruites comme circuits défectueux autonomes. Le solveur est l'autorité physique ; Flutter est l'autorité de présentation et d'interaction ; aucun des deux ne doit empiéter sur le rôle de l'autre.

FIN DU CAHIER DES CHARGES - DOCUMENT DIRECTEUR